

■ Read

`Read["file"]` reads one expression from a file, and returns the expression.

`Read["file", type]` reads one object of the specified type.

`Read["file", {type1, type2, ...}]` reads a sequence of objects of the specified types.

Possible types to read are:

<code>Byte</code>	a single byte, returned as an integer code
<code>Character</code>	a single character, returned as a one-character string
<code>Number</code>	an integer or an approximate number, given in "E" format
<code>String</code>	a string
<code>Real</code>	an approximate number, given in "E" format
<code>Expression</code>	a complete <i>Mathematica</i> expression

■ Objects of type `Real` can be given in the scientific notation format used by languages such as C and *fortran*. A form like `2.e5` or `2E5` can be used to represent the number 2×10^5 . Objects read as type `Real` are always returned as approximate numbers. Objects read as type `Number` are returned as integers if they contain no explicit decimal points. ■ Objects of type `String` must be terminated by newlines (line feeds or returns). ■ You can specify any nested list of types for `Read` to look for. Each successive object read will be placed in the next position in the list structure. A depth-first traversal of the list structure is used. ■ Example: `Read["file", {Number, Number}]` reads a pair of numbers from a file, and gives the result as a two-element list. ■ `Read["file", {{Number, Number}, {Number, Number}}]` reads a 2×2 matrix, going through each column, then each row. ■ If the file you specify is not already open, `Read` opens the file, and starts reading at the beginning. You can open a file for reading using `OpenRead`. ■ There is always a "current point" maintained for any file. When you read an object from a file, the current point is left after the input you read. Successive calls to `Read` can therefore be used to read successive objects in a file. ■ You can reset the current point to the beginning of a file by closing the file, and then reopening it. ■ `Read` returns `EndOfFile` for each object you try to read after you have reached the end of the file. ■ `Read` returns `Null` if it cannot read an object of the type you requested. ■ You can use `Read["!command"]` to read through a pipe from an external program. ■ See page 302. ■ See also: `Input`, `Get`.